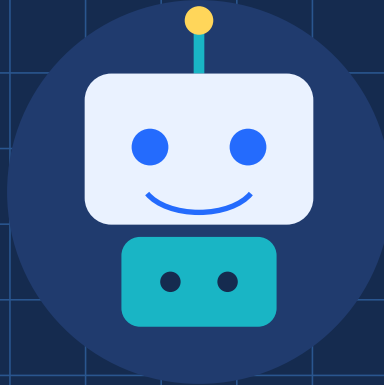


CRYSTAL DASH

Mission 1: Wake the Robot

Build your first Godot player and bring it to life with GDScript.



YOUR MISSION

Create a 2D robot, give it a collision shape, connect its controls, and drive it around a game window using WASD or the arrow keys.

NAME

DATE

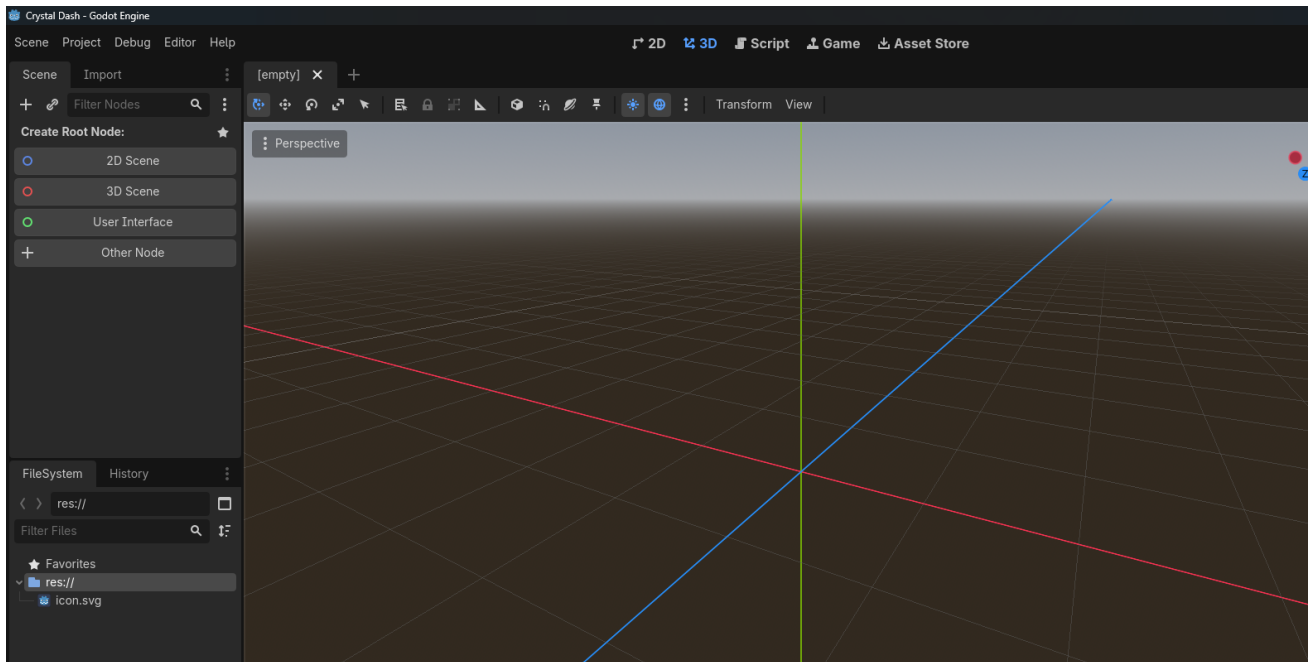
Tested with Godot 4.7.1 stable. Also compatible with Godot 4.7.2.

1. Start a new mission

Godot opens with an empty project. Unlike Scratch, it does not automatically create a stage and a sprite. We decide exactly what our game needs.

Create the project

1. Open Godot and choose Create New Project.
2. Name the project Crystal Dash.
3. Create a new empty folder named crystal_dash.
4. Choose the Compatibility renderer. This works well on lower-powered club computers.
5. Leave Version Control Metadata at its default and click Create & Edit.



The empty Godot editor. The highlighted 3D tab is harmless - we have not created a scene yet.

THINK ABOUT IT

Scratch gives every new project a Stage and a Sprite. Godot starts empty. What kinds of games might need something other than a traditional stage?

CHECKPOINT

You can see `res://` and `icon.svg` in the FileSystem panel.

2. Build the Player scene

A scene is a saved object or collection of objects. Our Player scene will hold everything the robot needs. Later, we can place it inside the main game scene.

Create the moving body

1. Click Other Node. Search for CharacterBody2D and click Create.
2. Rename the new node Player.
3. Press Ctrl+S and save it as `player.tscn`.

WHY CHARACTERBODY2D?

It is built for player characters that move around and collide with other objects. We get useful movement features without building our own physics system.

Give the Player an appearance

4. Select Player. Drag `icon.svg` from the FileSystem into the 2D workspace.
5. Godot creates a Sprite2D node named Icon. Rename it PlayerSprite.
6. Select PlayerSprite. Under Inspector > Transform, set Scale to x 0.35 and y 0.35.

What you see	What it means
Player	The friendly node name
CharacterBody2D	The node's actual type
PlayerSprite	Our name for the picture
Sprite2D	The picture node's actual type

SCENE TREE



THINK ABOUT IT

Why is it useful to give nodes clear names such as PlayerSprite instead of leaving every picture named Sprite2D?

3. Add collision and controls

The robot has a picture, but pictures do not collide. We need an invisible shape that Godot can use for physics.

Create the collision shape

1. Select Player and click + Add Child Node.
2. Add a CollisionShape2D.
3. In the Inspector, click Shape: <empty> and choose New CircleShape2D.
4. Click the words CircleShape2D to expand the resource. Set Radius to 22.

FAIR COLLISIONS

The collision circle is slightly smaller than the robot picture. Players usually find this fairer than losing when an enemy barely touches a transparent corner of the image.

SCENE TREE

● Player	CharacterBody2D
● PlayerSprite	Sprite2D
● CollisionShape2D	CollisionShape2D

Create named controls

5. Open Project > Project Settings > Input Map.
6. Add four actions: move_left, move_right, move_up, and move_down.

Action	Keyboard events
move_left	A and Left Arrow
move_right	D and Right Arrow
move_up	W and Up Arrow
move_down	S and Down Arrow

7. Expand each action, click + Add Event, and use Listening for Input to add both keys.

THINK ABOUT IT

Why tell the code to use an action named move_left instead of directly asking whether the A key is pressed? Hint: think about controllers, accessibility, and changing controls later.

CHECKPOINT

Your Player has a sprite, a collision shape, and eight keyboard events across four named actions.

4. Write your first GDScript

Now we connect the controls to the Player. GDScript looks a little like Python, but it is designed especially for Godot.

Attach the script

1. Select the root Player node.
2. Click Attach Script - the page icon with a green +.
3. Choose GDScript, select the Empty template, and use the path `res://player.gd`.
4. Click Create, then enter the code below exactly. Indentation matters.

```
extends CharacterBody2D

@export var speed: float = 250.0

func _physics_process(_delta: float) -> void:
    var direction: Vector2 = Input.get_vector(
        "move_left",
        "move_right",
        "move_up",
        "move_down"
    )

    velocity = direction * speed
    move_and_slide()
```

Code	Job
@export var speed	Creates the movement speed and shows it in the Inspector.
_physics_process	Runs again and again while the game is playing.
Input.get_vector	Combines four controls into one direction.
velocity	Stores direction multiplied by speed.
move_and_slide()	Moves the body and handles collisions.

THINK ABOUT IT

If direction is zero because no key is pressed, what value will velocity receive? What should the robot do?

CHECKPOINT

Press Ctrl+S. There are no red error markers beside the code.

5. Build the world and run

The Player is a reusable object. It now needs a main game scene to live inside.

Create Main

1. Press Ctrl+N, choose 2D Scene, and rename the root node Main.
2. Press Ctrl+S and save the scene as `main.tscn`.
3. Open Project > Project Settings > Display > Window. Set Viewport Width to 960 and Viewport Height to 540.
4. Select Main. Drag `player.tscn` from the FileSystem into the 2D workspace.
5. Select the new Player. Under Inspector > Transform > Position, enter x 480 and y 270.

SCENE TREE

● Main

Node2D

● Player

CharacterBody2D

WHY 480, 270?

The window is 960 pixels wide and 540 pixels tall. Half of 960 is 480, and half of 540 is 270, so this position is the exact centre.

Milestone: wake the robot

6. Save with Ctrl+S. Press F6 to run the current scene.
7. Click inside the game window. Test WASD, the arrow keys, and diagonal movement.

MILESTONE UNLOCKED

Your robot moves! You have created scenes, used three node types, configured input actions, written a script, and run a playable Godot project.

Quick fixes

Problem	Check
Robot does not move	Click the game window. Check the four action names and confirm <code>player.gd</code> is attached to Player.
Only part of the robot appears	Confirm Player position is x 480, y 270 in <code>main.tscn</code> .
Red script error	Compare spelling, quote marks, brackets, and indentation with the code on page 5.

6. Level up

Finished early? Experiment like a game designer. Change one thing at a time, predict what will happen, test it, and then record what you discovered.

LEVEL UP: Speed scientist

Select Player and find Speed in the Inspector. Try 120, 400, and one value of your own. Which feels best?

My best speed: _____ Why: _____

LEVEL UP: Choose a new spawn point

In `main.tscn`, change Player's Position. Predict where these coordinates will place it before testing:

(100, 100) Prediction: _____

(860, 440) Prediction: _____

LEVEL UP: Robot remix

Change PlayerSprite's Scale or replace its Texture with another image. Keep the collision circle sensible for the new picture.

Code detective

THINK ABOUT IT

1. What do you predict happens when Left and Right are held together?
2. Why does `Input.get_vector` stop diagonal movement being faster?
3. What advantage do we gain by saving Player as its own scene?

Scratch-to-Godot bridge

Scratch idea	Godot idea
Sprite	A scene made from nodes
When green flag clicked	Run the project or scene
Forever	<code>_physics_process()</code>
Key pressed?	Input actions
Set x / change x	Position and velocity

CHECKPOINT

MISSION COMPLETE

- [] My Player scene is saved.
- [] My controls work with arrows and WASD.
- [] My script has no errors.
- [] My robot moves in the Main scene.

NEXT MISSION

Build the crystal collectible, use a signal to detect pickups, and add a score to the screen.